

Path-Minimal Objects in ArkTS Symbolic Execution

From Path Constraints to TypeScript Tests

Aleksei Menshutin

ITMO University

Saint Petersburg, Russia

alex.menshutin99@gmail.com

Abstract

We study symbolic test generation over the ArkTS intermediate representation (ArkIR), obtained from ArkTS/TypeScript/JavaScript source or from ArkTS bytecode transformed in the ArkTS toolchain. Our contribution is *path-minimal objects* (PMOs): we construct inputs whose objects contain exactly the properties required to satisfy a chosen path, and emit runnable TypeScript tests (Jest-style) that replay the same path with lightweight structural assertions. This explicit minimality fixes the test's pre-state obligations, avoids noisy over-specified fixtures, and makes the generated inputs practical both for regression testing and for validating path-based alarms raised by other static analyses.

Our executor separates typing from path reasoning. Typing constraints collected from `typeof`/`instanceof` and available annotations are handled by a lightweight type solver that prunes forks and decodes unknown values, while the SMT path condition ranges only over numeric and boolean constraints plus field-membership atoms and guarded reads. Special run-time values (NaN, $-\infty$) and exceptions are handled precisely; in-path mutations and deletions never become entry obligations. On representative ArkIR fragments the engine automatically discovers and replays paths that exercise membership against absence, coercions to NaN, strict against loose equality, nested reads, write-then-delete, and unreachable branches. The evaluation is qualitative and demonstrates feasibility; expanded benchmarking and broader coverage of ArkTS instructions are left to future work.

1 Introduction

Programs in dynamically and gradually typed languages such as ArkTS and JavaScript/TypeScript routinely consume inputs whose kind and shape are not

fixed in advance: the same parameter may be a primitive value (number, string, boolean, null, undefined) or an object with an unknown set of properties. Control flow and observable behavior depends on membership tests, numeric and boolean coercions, short-circuit logic, deletions, and nested accesses prescribed by the ECMAScript semantics [3]. Symbolic execution is attractive for test generation in this setting because each explored path yields an explicit, quantifier-free path condition that characterizes feasibility [1, 6]. Two practical difficulties persist: (i) constructing correct constraints for such dynamic semantics (heap and alias modeling; creation and deletion of properties; strict and loose equality; coercions including $\emptyset$ truncation and `Number.isNaN`; short-circuiting; exception paths), and (ii) materializing solver models into *minimal* inputs without over-specifying objects or falsifying dynamic checks (for example, when a path hinges on `o.x + 1` yielding NaN).

Prior work on symbolic analysis and test generation for JavaScript exists but is comparatively sparse and specialized. It spans browser-focused security tools with string-heavy constraints, practical Node.js dynamic symbolic execution with regular-expression modeling, and trusted ES semantics with soundness guarantees. These directions advanced coverage and bug finding; yet they typically operate at the JavaScript source level and, by design and timeframe, do not leverage static typing information or ArkTS-level IR. Some also assume fixed input schemas or emit inputs larger than a particular path actually requires, which can make tests brittle and obscure the role of dynamic semantics.

Kudzu targeted client-side JavaScript security with concolic exploration and a specialized string solver, demonstrating automated input generation for vulnerability discovery in web applications [14]. Developed for browser JavaScript and predating the widespread use

of TypeScript and the emergence of ArkTS, it naturally did not address type-guided object-shape minimality or IR-level analysis.

ExpoSE advanced practical dynamic symbolic execution for Node.js, exploring server-side paths and modeling regular expressions while emitting executable tests [10]. It operates at the JavaScript source level and, consistent with its goals and timeframe, does not incorporate static type information (for example, TypeScript annotations) or ArkTS-level IR to guide input minimality.

Cosette pursued trusted ES semantics and soundness, enabling symbolic tests that either prove properties or yield violating inputs under a formal ES model [13]. The emphasis is semantic fidelity rather than IR-level execution or type-guided construction of path-minimal object inputs for test generation. Related but orthogonal are dynamic instrumentation (Jalangi) and static abstract interpretation (JSAI), as well as concolic efforts such as SymJS (2014) [5, 8, 15].

JaVerT 2.0 brings compositional symbolic execution for JavaScript on top of a mechanized, separation-logic-based semantics, targeting safety verification rather than test generation [4]. It operates at the JavaScript source level with strong reasoning principles, but does not address IR-level execution nor the construction of path-minimal object inputs. Li and Xie study *in-situ* concolic testing for JavaScript, executing instrumented code within the native runtime and developer workflow [9]. Their focus is practicality of concolic exploration at source level; PMO extraction and typing commitments are out of scope. Finally, Loring et al. provide a sound semantics for JavaScript regular expressions [11], which is complementary to our work: precise regex reasoning strengthens string-heavy analyses (including symbolic execution), whereas our case studies emphasize numeric semantics, membership/absence, strict against loose equality, and path-minimal witnesses.

We approach the problem at the intermediate-representation level. Our pipeline ingests ArkIR—the three-address, control-flow-graph representation produced from ArkTS, TypeScript, and JavaScript sources or obtained by transforming ArkTS bytecode with the ArkAnalyzer front end—and symbolically interprets it under JavaScript/TypeScript semantics [2, 3]. The interpreter is built on the Universal Symbolic Virtual Machine (USVM), whose solver-agnostic core (via KSMT) abstracts constraint solving and symbolic

memory, while a language plug-in implements ArkIR semantics [7, 16].

For each explored path the interpreter produces a quantifier-free path condition π together with a symbolic machine state that determines the observable outcome (a return value or an exception). The constraint ranges over theories matching ECMAScript runtime behavior: floating-point arithmetic for numbers (IEEE-754 doubles), booleans, and bit-vectors where truncation to 32-bit integers is required; the heap is modeled with uninterpreted sorts for object addresses and field identifiers. Crucially, we separate *typing constraints* from *field obligations*. Typing constraints (collected from `typeof/instanceof` and available annotations) are maintained alongside execution by a lightweight solver and used as *commitments* to prune infeasible forks and decode unknown values; *typing predicates are not injected into π* . By contrast, field obligations are represented *inside π* via field-membership atoms and guarded reads; these determine exactly which properties must be present in entry objects and are solved by the SMT back end independently of the typing solver. From π we then compute path-minimal inputs: when the path expects an object parameter o , a path-minimal object contains exactly the properties whose existence or value is required by the path; when the path accepts a primitive, the input is the corresponding primitive witness. Finally, we read concrete values from the solver model, serialize minimal TypeScript literals (or primitives), and generate runnable Jest-style tests; the default assertion is lightweight (structural equality), while richer oracles are left to future work.

Why PMOs matter. In dynamically typed code, tests frequently become brittle because fixtures *over-specify* input objects: extra properties can mask missing membership checks, make intended paths infeasible, or cause spurious failures after refactorings. PMOs address this by making the pre-state *explicit and minimal*: every key that appears in the test literal is justified by a field-membership atom or a guarded read in π , and nothing else is required. As a result, the generated tests are smaller, easier to interpret, and less sensitive to unrelated changes—developers can focus on the semantics under test rather than on incidental “noise” properties.

Although the case studies below highlight a small set of dynamic-semantics phenomena, the interpreter supports a broader subset of ArkTS/TypeScript constructs relevant to symbolic execution (property reads

and writes, membership tests, strict and loose equality, numeric coercions and arithmetic over IEEE-754 doubles, guarded nested accesses, deletion, exception edges, and control flow at the ArkIR level). Features outside this evaluation—such as symbolic property keys, prototype/Proxy semantics, and functional fields—are compatible with our design but left to future empirical study; we outline the scope in Section 2.

Intended use. We currently target two closely related workflows. First, regression-oriented test generation for ArkTS/ArkIR code. During exploration, the symbolic executor discovers feasible paths and may also expose failing behaviors (for example, unexpected exceptions or assertion violations). For any such path, we can materialize a path-minimal witness and emit a small TypeScript test that replays exactly this behavior. In this setting, the symbolic engine itself serves both as the path explorer and as the generator of a regression suite that can be re-run independently of the symbolic infrastructure.

Second, path-feasibility checking for external analyses: given a candidate trace or alarm produced by another tool (for instance, a static analyzer that reports a path to a potential issue), we can symbolically re-interpret the trace on ArkIR and either show it to be infeasible or materialize a path-minimal witness that demonstrates how the path can actually execute. In both workflows, path-minimality is what makes the generated inputs practical to inspect: developers see exactly the properties that matter for the reported path, and nothing else. This is particularly useful for building regression suites (where failing inputs should stay as small and stable as possible) and for inspecting alarms from other analyses, where large, automatically constructed fixtures would otherwise obscure the underlying behavior. A quantitative study of these use cases is left to future work; in this paper we focus on establishing that such witnesses can be constructed reliably at the ArkIR level.

Contributions. (1) An ArkIR-level symbolic interpreter for ArkTS/TypeScript/JavaScript that preserves dynamic ECMAScript behaviors while exploiting available type information without polluting the path condition. (2) The notion and construction of *path-minimal objects* (PMOs) that contain exactly the properties required by a chosen path. (3) An implementation that emits executable TypeScript tests and, on representative ArkIR fragments, reproduces membership against absence, coercions to NaN, strict against loose equality,

guarded nested reads, write-then-delete, correlated integer truncations ($|0$), and unreachable branches. Comprehensive benchmarking and expanded coverage of ArkTS instructions (for example, modules, enums) are left to future work.

2 Methods

This section presents the end-to-end pipeline from source code or ArkTS bytecode to executable TypeScript tests. We first fix the setting (ArkIR and semantics), then describe the symbolic interpreter, the logical theories and our treatment of typing constraints, define path-minimal inputs, and finally detail the construction from a path condition to tests.

2.1 Pipeline Overview

Given an entry function implemented in ArkTS, TypeScript, or JavaScript (or compiled to ArkTS bytecode), we *enumerate feasible terminal paths* (normal returns and exceptions) in its ArkIR and produce a *suite of tests*, one per path, subject to stopping criteria. The workflow is:

- (1) **Front end.** Parse or load the program and produce ArkIR, a three-address, control-flow-graph representation [2].
- (2) **Symbolic exploration.** Execute ArkIR symbolically under JavaScript/TypeScript semantics, forking on branches and maintaining a worklist; for each terminal path, accumulate its quantifier-free path condition π and the observed outcome [3].
- (3) **Typing alongside.** Run a lightweight, SMT-backed type-constraint solver alongside execution; it propagates TypeScript/ArkTS typing information but does not inject typing predicates into π .
- (4) **Per-path solving.** For *each* terminal path, submit its π via USVM/KSMT to obtain a model M when satisfiable [7, 16].
- (5) **Per-path minimal input extraction.** From π and M , derive a path-minimal input W (object PMO or primitive), separating entry obligations from in-path mutations.
- (6) **Test generation (optional).** Serialize each W as a minimal TypeScript literal (or primitive). If desired, generate runnable Jest-style tests—one per terminal path—that invoke the entry point. Special run-time values are preserved in serialization; assertions are lightweight structural checks, as the focus is on faithful input materialization.

2.2 Front End and ArkIR

Inputs are either source files (.ts, .js, .ets) or ArkTS bytecode (.abc). ArkTS is a gradually typed language in the TypeScript/JavaScript family; source files are compiled to ArkTS bytecode. We use ArkAnalyzer to translate any of these inputs into ArkIR, a three-address IR with basic blocks and explicit control-flow edges [2]. Regardless of origin (source or bytecode), ArkIR has the same instruction set, which increases the applicability of our approach. Although the bytecode itself is untyped, ArkIR supports optional type annotations; when ArkIR is produced from typed sources, available types are carried into IR and exposed to our pipeline. When types are absent or intentionally unknown, ArkIR marks them explicitly as `EtsUnknownType`; our pipeline operates both with and without such annotations (see Section 2.4 for the typing-constraint treatment).

Front-end limitations. ArkAnalyzer is under active development and currently exhibits several limitations relevant to our setting: the JavaScript `in` operator is not emitted (the front end lowers it to `undefined` in IR), optional-chaining reads are unavailable, there are issues with module imports, and a few other restrictions persist. To avoid confounding front-end bugs with execution semantics, *for the case studies in this paper we construct ArkIR manually* using an embedded Kotlin DSL that mirrors the IR classes. Instead of the Node-based front end serialising IR, we instantiate the corresponding Kotlin objects directly and feed them to the interpreter.

2.3 Symbolic Interpretation on USVM

We execute ArkIR on top of the Universal Symbolic Virtual Machine (USVM). A symbolic state is $\langle \ell, \sigma, \pi \rangle$, where ℓ is the current ArkIR instruction in the CFG, σ is the symbolic memory (stack and heap), and π is the path condition. The interpreter uses a priority worklist; scheduling comes from the USVM core and mixes heuristics with randomness to avoid stagnation (for example, in loops). Each step dequeues the highest-priority state, executes one ArkIR instruction, and enqueues zero or more successors. Exploration stops when the queue is empty or a stopping criterion is met (time limit, coverage target, or a designated location). In our case studies we use breadth-first exploration; on larger projects we rely on the default “closest-to-uncovered (randomized)” policy with fast satisfiability probes and per-query timeouts.

State and memory. Object identities range over an uninterpreted sort `Addr`. The heap follows the USVM regioned memory model and can be seen as families of arrays indexed by `Addr`, enabling alias reasoning through equalities over addresses together with standard store/select axioms. We keep a strict separation between *entry obligations* (requirements on the initial input) and *in-path mutations* (writes and deletions performed during the path): the latter do not become input obligations. Values of primitive types (for example, numbers and booleans) live in symbolic stack-frame registers and are passed by value; only reference-like values inhabit the heap and are addressed via elements of `Addr`. Logical domains used by constraints are listed in Section 2.4.

Unknown inputs and EtsFakeRef. Inputs may arrive with ArkIR type `EtsUnknownType`. To avoid early path explosion we represent an unknown with an *EtsFakeRef* that encapsulates alternatives (in particular, number, boolean, reference-like, etc). In the SMT encoding, the choice among these alternatives is guarded *only* by runtime-kind atoms that live in the path condition π —`isNumber(x)`, `isBoolean(x)`, `isRef(x)`—via nested `ite`. Because SMT branches must share a sort, heterogeneous results are lifted to a uniform reference-like carrier and decoded when needed. If `isRef(x)` holds, subsequent reasoning about the concrete class (subtype/nominal hierarchy, `instanceof`, and dispatch regions) proceeds in the separate typing component τ maintained by our type solver. These class/subtype constraints are *not* encoded into π , are not used to guard the *EtsFakeRef* `ites`, and never assert field membership. Field requirements are introduced independently in π by membership atoms $H(o, k)$ and the corresponding guarded reads.

Property reads, membership, writes, and deletion. For a read `getprop (v := o[k])` we first guard exceptional cases: if o is `null` or `undefined`, the path takes an exceptional edge recorded as the outcome. Otherwise, let $addr(o) \in \text{Addr}$ and let $fid(k)$ be the field identifier for key k . We model membership by a boolean atom $H(o, k)$ that denotes $fid(k) \in \text{Fields}(addr(o))$. The read yields

$$v \equiv \text{ite}(H(o, k), \text{Heap}_k(addr(o)), \text{undefined}),$$

and if v is used in predicates we add the corresponding atoms to π . A membership test asserts $H(o, k)$ or $\neg H(o, k)$. A write `setprop` updates Heap_k at $addr(o)$;

a deletion `delprop` clears membership for (o, k) by updating *Fields*. These effects are in-path and do not add entry obligations.

Calls and control flow. A call transfers control to the callee’s CFG and back; the symbolic stack in σ receives a new frame, and a return writes the result to the caller frame and continues after the call. These steps mirror the concrete interpreter, but over symbolic values.

Virtual calls and target resolution. If the callee is statically known (the IR names the defining class), we jump to that CFG directly. Otherwise we resolve virtual dispatch from the (symbolic) type stream of x . Let $\text{Def}(\text{foo})$ denote the set of classes that define or override `foo`. For $T \in \text{Def}(\text{foo})$ define

$$\text{Over}(T) := \{ D \in \text{Def}(\text{foo}) \mid D \text{ is a subtype of } T \},$$

i.e., classes that override `foo` strictly below T . The dispatch region where the call resolves to the implementation in T is

$$R_T(x) = \text{Sub}(x, T) \wedge \bigwedge_{D \in \text{Over}(T)} \neg \text{Sub}(x, D).$$

Intuitively, x must be a subtype of T but not a subtype of any strict subtype of T that also overrides `foo`. For each candidate T we fork a successor, extend the *typing component* of the path condition with $R_T(x)$, and continue execution in the CFG of $T.\text{foo}$.

We track these typing constraints alongside the numeric/boolean/heap part of the path condition and use a lightweight type solver to rule out type-inconsistent forks and resolve virtual calls. We make this split precise in Section 2.4.

Equality and coercions. For reference-like values, both `===` and `==` use address equality (same *Addr*). For primitives, strict equality uses same-type and value equality, while loose equality follows the ECMAScript coercion rules. Numeric comparisons apply *ToNumber* to primitives; `null == undefined` holds; `NaN` is not equal to itself; `Number.isNaN` is a predicate over floating-point that is true exactly for `NaN`. Numeric *add* applies *ToNumber*; string concatenation is out of scope. Each case contributes atoms to π consistent with the ECMAScript specification.

Branching, pruning, and exceptions. At a conditional ArkIR edge with boolean guard c —that is, the value of the edge’s condition expression—we fork states with path conditions $\pi \wedge c$ and $\pi \wedge \neg c$. Before enqueueing each successor, we issue a fast check-sat probe via

KSMT; infeasible forks are discarded. The USVM invariant is that queued states are feasible under the current model unless a fork has just occurred. Instructions that may raise exceptions (for example, reading a property of `null` or `undefined`, or domain-specific checks such as disallowing negative indices) generate a dedicated exceptional successor; the exception (type/value) is recorded as the path outcome for test emission.

2.4 SMT Interface, Typing Constraints, and Field Obligations

Constraint language. Path conditions are quantifier-free and range over theories that match JavaScript and TypeScript run-time behavior: (i) floating-point arithmetic for numbers (IEEE-754 doubles, including `NaN`, $\pm\infty$, and signed zero); (ii) booleans; (iii) 32-bit bit-vectors for integer truncation where required by the IR. Heap reasoning uses uninterpreted sorts for object addresses (*Addr*) and field identifiers; arrays (*store/select*) provide per-field maps from *Addr* to values, and a field-membership map records which keys are present at a given address. All constraints in π are ground and may use `ite`; no quantifiers are introduced.

Two coordinated channels of constraints. We view the path condition as a pair $\langle \pi, \tau \rangle$: π is the SMT-solved numeric/boolean/heap part, while τ is a separate *typing component* accumulated during execution.

From an expressiveness point of view, nothing prevents us from encoding all typing predicates directly into the SMT path condition π . We avoid this for performance reasons: in symbolic execution, SMT solving is already the dominant cost, and pushing the full typing discipline (class hierarchies, subtype predicates, dispatch regions) into π would further increase the size and structural complexity of the formulas that the solver must handle.

Instead, for each typing atom that arises during execution we introduce a fresh propositional variable in π and maintain a separate, lightweight typing component τ that records the corresponding class/subtype constraints and decides which combinations of these variables are actually type-consistent. The SMT solver only sees the propositional skeleton of typing constraints intertwined with numeric and heap reasoning, while the concrete class/subtype relationships are resolved by the external type solver. This design keeps the SMT side focused on heap membership and arithmetic—where

its power is most needed—while offloading type reasoning to a cheaper, specialized component without compromising the overall soundness of path feasibility.

(A) *Typing constraints split across π and τ .* A small set of run-time-kind predicates $\text{isNumber}(x)$, $\text{isRef}(x)$, $\text{isBoolean}(x)$ is encoded *inside* π solely to guard the ite -encoding of EtsFakeRef and select which alternative is taken. In contrast, all *class/subtype* information (from typeof/instanceof , available annotations, and dispatch-region predicates $R_T(x)$) is recorded in the separate typing component τ and solved by a lightweight type solver; the solver’s commitments prune infeasible forks and resolve virtual calls when $\text{isRef}(x)$ holds. These class/subtype constraints are not translated into π . Crucially, neither the kind atoms in π nor the commitments in τ introduce or modify field-membership atoms $H(o, k)$; they do not affect field minimality or the set of required properties in PMOs.

(B) *Field membership and guarded reads (encoded in π).* Field obligations arise solely from program use of properties along the path. For a read $\text{getprop}(v := o[k])$ we introduce a membership atom $H(o, k)$ and the guarded read

$$v \equiv \text{ite}(H(o, k), \text{Heap}_k(\text{addr}(o)), \text{undefined}).$$

Explicit membership tests assert $H(o, k)$ or $\neg H(o, k)$; writes and deletions update per-field arrays and the membership map but do *not* add entry obligations. Because we assert $H(o, k)$ only when presence is required or the read value is constrained, and because membership is represented with USVM’s symbolic sets that have an inclusion-minimizing interpretation, the resulting field set $\text{Fields}(\text{addr}(o))$ that satisfies π is *minimal by construction*: every asserted $H(o, k)$ is necessary for the path, and all non-required memberships remain false in the model.

Solver interaction. We access the SMT solver via KSMT (Z3 in our setup) with shallow push/pop: conditional forks are checked by a fast check-sat probe (per-query timeout), feasible forks are enqueued. On terminal paths (return or exception) we obtain a model M and read concrete values, preserving run-time corner cases (the fact that $\text{NaN} \neq \text{NaN}$, signed zero, infinities). Store/select axioms ensure that reads observe the latest write for a given $(\text{Addr}, \text{field})$ pair.

From model to witnesses (fields independent of typing). Given $M \models \pi$, we compute the path-minimal

input for an entry object o by

$$\text{dom}(W_o) = \{ k \mid M \models H(o, k) \wedge \text{req}(o, k) \}.$$

Here $\text{req}(o, k)$ holds iff along the explored path either (i) a read of $o[k]$ is used in a predicate in π , or (ii) there is an explicit membership test on k . Writes and deletions do not make $\text{req}(o, k)$ true. Values for required keys are taken from the per-field arrays in M ; if $M \models \neg H(o, k)$ then $o[k]$ is undefined and $k \notin \text{dom}(W_o)$. Typing commitments neither add nor remove elements of $\text{dom}(W_o)$; they only select which alternative of an EtsFakeRef is realized in the encoding.

2.5 Path-Minimal Inputs

This subsection gives the core construction of path-minimal inputs used throughout the paper. We use the term *witness* for any concrete path-minimal input (object or primitive), and *path-minimal object (PMO)* when the entry is an object.

Goal. Given a satisfiable path condition π for an entry point, we construct a concrete input that is *just enough* to replay the chosen path and produce the same outcome. A step-by-step instantiation of this construction on a concrete example (function f3b) is given in Section 2.6.

Objects. Intuitively, a path-minimal input for a given path is the smallest concrete input that still forces the interpreter to follow exactly this path and produce the same observable outcome. For object parameters we would like to keep only those properties that the path actually depends on (via membership tests or value constraints), and drop everything else – including fields that are written and later deleted inside the path.

Let o be an entry object and let $M \models \pi$ be a (fixed) model for the path. We write $H(o, k)$ for the membership atom introduced by a read or an explicit membership test on key k (Section 2.3), and $\text{req}(o, k)$ for the fact that along the explored path either (i) a read of $o[k]$ participates in a predicate in π , or (ii) there is an explicit membership check on k . Intuitively, $\text{req}(o, k)$ records that the presence or value of $o[k]$ can influence feasibility of the path.

As defined in Section 2.4, the domain of the concrete witness for o is

$$\text{dom}(W_o) = \{ k \mid M \models H(o, k) \wedge \text{req}(o, k) \}.$$

For each $k \in \text{dom}(W_o)$ the value $W_o[k]$ is read from the per-field array in the model M ; if $M \models \neg H(o, k)$,

then $k \notin \text{dom}(W_o)$ and $o[k]$ evaluates to undefined on replay.

Primitives. If the entry is a primitive (number, boolean, string, null, undefined), the witness is the corresponding value read from M (with precise handling of NaN, signed zero, and infinities).

Correctness and minimality properties. The construction above enjoys the following properties for any satisfiable path condition π and any model $M \models \pi$:

- (i) *Satisfiability.* By definition $M \models \pi$, hence the witness input satisfies the path condition.
- (ii) *Replay correctness.* Evaluating ArkIR from the entry with the witness follows the same path and produces the same return or exception, because guarded reads and membership atoms are preserved.
- (iii) *Field minimality.* For any $k \in \text{dom}(W_o)$, removing k from W_o falsifies the path (the guarded read becomes undefined or a required membership is dropped), so W_o contains only necessary keys.
- (iv) *Mutation exclusion.* Properties created and later deleted on the path do not enter $\text{dom}(W_o)$; in-path writes and deletions never become entry obligations.
- (v) *Typing independence.* Typing commitments select alternatives inside encodings (for example, `EtsFakeRef`) but do not add or remove keys from $\text{dom}(W_o)$; field minimality is independent of the typing solver.
- (vi) *Address α -invariance.* Renaming object addresses in M does not change $\text{dom}(W_o)$ nor the witness values.

2.6 From π to Path-Minimal Inputs and Runnable Tests

Building on the definition and properties in Section 2.5, we now give a concrete algorithm that turns a satisfiable path condition into a path-minimal witness and an optional runnable test.

Inputs. A terminal path with its path condition π , the set of entry variables (objects and/or primitives), and an SMT model M with typing commitments from the USVM type solver (Section 2.4).

Algorithm.

- Step 1:** *Solve.* Invoke the SMT back end via KSMT on π ; if satisfiable, obtain a model M .
- Step 2:** *Extract witnesses.*
 - a. For each entry object o , compute $\text{dom}(W_o) = \{k \mid M \models H(o, k) \wedge \text{req}(o, k)\}$ and read $W_o[k]$ from M . Decode alternatives (for example, inside `EtsFakeRef`) using typing

commitments; preserve corner cases (NaN, $-\infty$, $+\infty$, undefined/null).

- b. For each primitive entry, read its concrete value from M with the same corner-case handling.

Step 3: *Emit minimal literals.* Serialize each W_o as a minimal TypeScript object containing exactly the keys in $\text{dom}(W_o)$; serialize primitives directly. Preserve special values during serialization (for example, represent NaN and $-\infty$ explicitly).

Step 4: *(Optional) runnable test.* When desired, generate a Jest-style test that constructs the serialized inputs, invokes the entry point, and applies a lightweight structural assertion (implemented via JSON-based comparison). This default check is not a substitute for full semantic equality; richer semantic assertions are left for future work.

Step 5: *Persist.* Output one witness per terminal path and, optionally, its runnable test; we do not deduplicate witnesses in this work.

Replay guarantee. Because membership atoms and guarded reads that mention entry values are satisfied by M , and because emitted literals include exactly the required keys with the values from M , executing the generated test under the same semantics replays the chosen path and reproduces its outcome.

Special values and environment. Serialization preserves NaN, $+\infty$, $-\infty$, and $\pm\infty$ as well as undefined/null, matching the semantics assumed during symbolic execution. Runnable tests target a standard TypeScript (Node) environment. The default JSON-based assertion is a convenience check and may not distinguish FP corner cases (for example, NaN, $-\infty$); improving these oracles is left for future work.

Example of construction. We now instantiate the construction from Section 2.5 on a concrete example, the function `f3b` from Listing 2. There are three terminal paths: (i) an exceptional path when the whole argument is undefined; (ii) the return 1 path where `Number.isNaN(o.x + 1)` holds; and (iii) the return 2 path where the guard is false (see also Appendix A, Listing 8).

For the return 1 path, the ArkIR contains a read of `o.x` and a numeric addition `+1`. The guard $g(y) = \text{Number.isNaN}(y)$ is applied to the result. Symbolically, the path condition π contains constraints stating that

the guard is true and that the addition follows the ECMA Script floating-point semantics; in other words, it enforces that the value corresponding to y (that is, $v + 1$) is NaN, but there is *no* explicit membership test on x . At the heap level we still introduce the membership atom $H(o, "x")$ and a guarded read

$$v \equiv \text{ite}(H(o, "x"), \text{Heap}_{"x"}(\text{addr}(o)), \text{undefined}),$$

but π only constrains the result of $v + 1$ to be NaN. A satisfying model M can therefore choose the inclusion-minimal heap where $M \models \neg H(o, "x")$ and the coerced value of $o.x$ is undefined, making $o.x + 1$ evaluate to NaN. By the definition of $\text{dom}(W_o)$, no key is both required and present in M , so $\text{dom}(W_o) = \emptyset$ and the path-minimal witness is the empty object literal $\{\}$. Non-minimal inputs such as $\{x: \text{undefined}\}$ also satisfy π , but they contain an extra, unnecessary property and are therefore not PMOs.

For the return 2 path, the engine produces a model where the guard is false. One concrete witness, also used in the generated tests, is $\{x: \text{null}\}$: here $M \models H(o, "x")$, the value of $o.x$ is constrained via the guard, and $\text{req}(o, "x")$ holds because the read participates in π . Consequently $\text{dom}(W_o) = \{"x"\}$ and the PMO contains exactly this single key.

In all three cases the generated witnesses, when serialized as TypeScript literals and plugged into the auto-generated tests in Appendix A, replay their respective paths under the interpreter.

Artifact. A reproducible artifact accompanies this paper and contains the implementation of our ArkIR-level symbolic executor, the path-minimal object extraction pipeline, and the case-study IR fragments used in Section 3. The artifact is available on Zenodo [12].

3 Results

We evaluate the prototype on small ArkIR fragments that mirror characteristic JavaScript/TypeScript idioms. The goal is to demonstrate that the engine emits *path-minimal* inputs which reliably replay specific paths.

USVM configuration. For all results reported here, the solve procedure ran *without a timeout*; the absence of a timeout is important for FP + bit-vector mixes such as f5.

Coverage metric. The symbolic engine maintains ArkIR-level instruction coverage during exploration. Whenever a terminal path is processed and its witness is materialized (optionally with a runnable test), all ArkIR instructions on that path are marked as covered.

Method coverage is computed as the ratio of covered to total ArkIR instructions in that method; no external tooling is used.

Representative cases with minimal witnesses

Membership against absence. Listing 1 shows the simplest shape reasoning. For f1 (" x " in o && $\text{typeof } o.x === \text{"number"}$) the minimal success witness contains exactly one key x with a numeric value; the failure is the empty object; a separate terminal path is the expected exception on an undefined entry (due to a base property read). For f2 the situation is dual: success is the empty object; failure is any object that has x ; the exception path is present as well.

```
function f1(o: any) {
  if ("x" in o && typeof o.x === "number") return 1;
  return 2;
}
function f2(o: any) {
  if (!("x" in o)) return 1;
  return 2;
}
```

Listing 1. Membership and absence

Numeric guard and NaN via coercion. In f3a (Listing 2) success requires that $o.x$ is numeric and $o.x + 1 > 0$. In f3b, $\text{undefined} + 1$ evaluates to NaN, so the path-minimal witness for the return 1 branch is the empty object $\{\}$: along this path $o.x$ is read but not required to be present, so it evaluates to undefined and $o.x + 1$ yields NaN. Non-minimal inputs such as $\{x: \text{undefined}\}$ also drive the same branch but introduce an unnecessary property and are therefore not PMOs. A non-NaN path (for example, $\{x: \text{true}\}$) leads to return 2. As before, passing undefined as the whole argument produces a distinct exception path.

```
function f3a(o: any) {
  if (typeof o.x === "number" && o.x + 1 > 0) return 1;
  return 2;
}
function f3b(o: any) {
  const y = o.x + 1;
  if (Number.isNaN(y)) return 1;
  return 2;
}
```

Listing 2. Numeric guard and NaN via coercion

Auto-generated tests for f3b are provided in Appendix A (Listing 8).

NaN by self-inequality. Listing 3 captures the floating-point corner where $x \neq x$ holds iff x is NaN. The minimal success is $\{x: \text{NaN}\}$; a tiny positive subnormal (for example, $4.9\text{E-}324$) drives the failure branch; the exception path appears when the base is not an object.

```
function f11_nan(o: any) {
  if (typeof o.x === "number" && o.x !== o.x) return
    1;
  return 2;
}
```

Listing 3. NaN detection by self-inequality

Strict against loose equality; in-branch mutation. Listing 4 contrasts strict and loose equality. For `f6_strict` the minimal success witness is $\{x: -0.0\}$ (signed zero satisfies $=== 0$). For `f7_loose` any value coercible to 0 suffices; the smallest is $\{x: \text{false}\}$. The assignment `o.x = 124` occurs *inside* the successful path and, by design, does not become an input obligation.

```
function f6_strict(o: any) {
  if (o.x === 0) return 1;
  return 2;
}
function f7_loose(o: any) {
  if (o.x == 0) {
    o.x = 124; // in-path mutation; not an
               // input obligation
    return 1;
  }
  return 2;
}
```

Listing 4. Strict against loose equality; in-branch mutation

Nested property requirements. In `f9` the success condition is $o.x.y === \text{true}$. The engine produces the smallest nested shape $\{x: \{y: \text{true}\}\}$, with no superfluous keys at either level; Listing 5 shows the code.

```
function f9(o: any) {
  if ("x" in o && o.x && "y" in o.x && o.x.y ===
    true) return 1;
  return 2;
}
```

Listing 5. Nested property requirements via explicit membership checks

In `f10_optchain` we express a similar pattern using a guarded nested read without spelling out the full

membership chain. The code first binds $x = o.x$ and then checks $x \neq \text{null} \ \&\& \ x.y === 1$. From the PMO construction point of view, the witness is again the smallest object that satisfies the guard, here $\{x: \{y: 1\}\}$, but the obligation now comes from a guarded read rather than an explicit "y" in x test in the source; Listing 6 illustrates this variant.

```
function f10_optchain(o: any) {
  const x = o.x;
  if (x != null && x.y === 1) return 1;
  return 2;
}
```

Listing 6. Nested property requirements: guarded nested read

Delete after write; correlated coercions. Listing 7 shows two stressors. In `f4` we write `o.x`, delete it, and then check absence; symbolic execution discovers that the branch `return -1` is *unreachable*. Coverage for this method is therefore reported below 100% (we observed 82%), which is expected: the solver proves one syntactic branch infeasible. Crucially, the pre-state witness for the successful path stays empty—an in-path write followed by delete does not create an input obligation. In `f5` we require $(\text{Number}(o.a)|0) + (\text{Number}(o.b)|0) = 0$ while enforcing $o.a \neq o.b$. A minimal success uses one genuine numeric zero and another value that coerces to zero but is strictly unequal at runtime; for example, $\{a: 0, b: \{\}\}$. Because `f5` combines floating-point $\rightarrow$ 32-bit coercions with arithmetic, short SMT time limits can yield unknown; we therefore run the terminal-path solve without a timeout for this case (and, in general, for witness extraction).

```
function f4(o: any) {
  o.x = 1;
  delete o.x;
  if (o.x !== undefined) return -1; // unreachable
  return 1;
}
function f5(o: any) {
  const ai = (Number(o.a) | 0);
  const bi = (Number(o.b) | 0);
  const s = ai + bi;
  if (s === 0 && o.a !== o.b) return 1;
  return 2;
}
```

Listing 7. Delete after write; correlated coercions

Takeaways. Across all cases, the generated witnesses—and their runnable tests when emitted—replay the intended paths under our interpreter, and the witnesses are *minimal by construction*: every key present in a witness is justified by a required membership/read along the path, while keys never demanded by the path remain absent. The engine also flags unreachable branches (for example, f4), which explains sub-100% coverage in some methods without contradicting correctness.

Instruction coverage beyond these examples. Beyond the snippets, the interpreter covers a broad subset of ArkIR. In short, we handle:

- **Arrays:** allocation, dynamic indexing/growth, and common methods (for example, push, pop, slice, includes).
- **Scalars and bitwise:** IEEE-754 arithmetic; ordered comparisons; bitwise ops (`|`, `&`, `^`) and shifts.
- **Logic and control:** boolean and short-circuit logic; nullish coalescing; `typeof`, unary `-`, and `void`.
- **Objects/fields:** creation and updates (including nested), reads/writes, membership, and delete.
- **Calls:** direct, virtual, and overloaded calls; constructors; varargs; closures capturing locals/parameters.
- **Exceptions and types:** throw and conditional throws; structural equality; `instanceof` and virtual dispatch.
- **Asynchrony:** `async/await` over resolved and rejected promises.

While this coverage is sufficient to run real projects end-to-end, we deliberately do not present quantitative benchmarking here. Our aim in this paper is qualitative: to show that path conditions yield *path-minimal* witnesses and executable tests. A systematic evaluation on larger, real-world ArkTS/TypeScript codebases—together with performance measurements and comparative baselines—is left for future work. As noted earlier, full module `import/export` handling and precise support for enum fields remain open items; their absence can lead to conservative over-approximation in some cases and is an important direction for the next iteration of the tool.

4 Discussion

Why path-minimal objects matter. As outlined in the introduction, path-minimal objects (PMOs) make tests explicit about their *pre-state*: each generated case includes exactly the properties a chosen path requires

and nothing else. This reduces fixture size and brittleness, avoids accidental dependence on surplus fields, and focuses attention on the predicates that drive behavior (for example, membership against absence, NaN, `-0`, guarded nested reads). Here we focus on how this plays out in practice and what it buys in everyday engineering scenarios such as CI and alarm triage.

Operating at the ArkIR level. Working over ArkIR provides a uniform target for inputs originating from either source or transformed bytecode, while staying faithful to ECMAScript execution. The IR level is also where available type information can be exploited without polluting the path condition. In our design, kind tests (for example, `isNumber`, `isBoolean`, `isRef`) steer `EtsFakeRef` encodings, whereas class/subtype constraints guide virtual dispatch and fork pruning. This separation lets us reason cleanly about heap membership and aliasing at the logic level and about run-time kinds and dispatch at the typing level.

What we observed empirically. On representative ArkIR fragments, the engine automatically discovered and replayed paths exhibiting membership against absence, coercions to NaN, strict against loose equality (`===/==`), nested reads, write-then-delete, correlated integer truncations (`| 0`), and unreachable branches. The built-in coverage accounting over ArkIR confirmed full instruction coverage for all demonstrated functions except one with a solver-proved unreachable branch, where coverage was correctly below 100%. These observations are preliminary and are meant to illustrate how path-minimal witnesses behave on characteristic ArkIR fragments.

Relation to existing tools. Prior JavaScript symbolic execution work has targeted browser security, Node.js servers, and formal ES semantics at the source level. Those systems advanced coverage and bug finding, but by design they do not leverage ArkIR or ArkTS typing, and several assume fixed input schemas or over-materialize objects. Our approach differs in two respects: (i) it operates at ArkIR with ECMAScript semantics while reusing available type information, and (ii) it constructs PMOs so that tests capture only path-required fields. Because of these representational differences, we did not pursue side-by-side comparisons; a fair baseline would require aligning environments and semantics, which is outside the scope of this initial study.

Limitations and threats to validity. The current evaluation is not a large-scale benchmark. Some language features are intentionally out of scope or

only partially modeled (for example, module imports/exports, enums, prototype/Proxy effects, symbolic property keys, and large string-heavy paths). Path explosion on sizable programs and the cost of solving mixed numeric/bit-vector constraints remain practical concerns. The generated harnesses are deliberately lightweight; while they run and compare results, richer checks (for example, `Number.isNaN`, `Object.is` for signed zero, explicit exception assertions) would strengthen behavioral matching. Finally, our focus on ArkIR means that host-environment APIs beyond the ES core are not modeled in this iteration.

Applications and engineering payoff. PMO-based tests are a good fit for CI: minimal fixtures make regressions easier to triage and reproduce, and the explicit pre-state helps pinpoint the semantics behind a failing path. As outlined in the introduction, two immediate applications are regression-style test generation and path-feasibility checking for external analyses. Witnesses can seed fuzzers, serve as starting points for delta debugging/minimization, and support differential testing across Ark versions or configurations. The same machinery can aid program understanding by surfacing exactly which fields and values toggle specific branches.

Future work. We plan to (i) expand ArkTS instruction coverage (including modules and enums), (ii) conduct comprehensive benchmarks on real-world projects, (iii) improve exploration with path prioritization, incremental solving, and deduplication of near-duplicate witnesses, and (iv) strengthen the generated harnesses with semantics-aware comparisons where needed. As ArkAnalyzer matures, tighter integration will also reduce manual IR construction in case studies and broaden empirical scope.

References

- [1] Cristian Cadar and Koushik Sen. 2013. Symbolic execution for software testing: three decades later. *Commun. ACM* 56, 2 (Feb. 2013), 82–90. <https://doi.org/10.1145/2408776.2408795>
- [2] Haonan Chen, Daihang Chen, Yizhuo Yang, Lingyun Xu, Liang Gao, Mingyi Zhou, Chunming Hu, and Li Li. 2025. ArkAnalyzer: The Static Analysis Framework for OpenHarmony. arXiv:2501.05798 [cs.SE] <https://arxiv.org/abs/2501.05798>
- [3] ECMA International. 2024. ECMAScript 2024 Language Specification. Online resource. <https://262.ecma-international.org/15.0/> Standard ECMA-262, 15th Edition, June 2024.
- [4] José Fragoso Santos, Petar Maksimović, Gabriela Sampaio, and Philippa Gardner. 2019. JaVerT 2.0: compositional symbolic execution for JavaScript. *Proc. ACM Program. Lang.* 3, POPL, Article 66 (Jan. 2019), 31 pages. <https://doi.org/10.1145/3290379>
- [5] Vineeth Kashyap, Kyle Dewey, Ethan A. Kuefner, John Wagner, Kevin Gibbons, John Sarracino, Ben Wiedermann, and Ben Hardekopf. 2014. JSAI: a static analysis platform for JavaScript. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Hong Kong, China) (FSE 2014). Association for Computing Machinery, New York, NY, USA, 121–132. <https://doi.org/10.1145/2635868.2635904>
- [6] James C. King. 1976. Symbolic execution and program testing. *Commun. ACM* 19, 7 (July 1976), 385–394. <https://doi.org/10.1145/360248.360252>
- [7] KSM T developers. 2023. KSM T: Kotlin/Java API for various SMT solvers. <https://ksmt.io/>. Kotlin/Java API and infrastructure for SMT solvers.
- [8] Guodong Li, Esben Andreasen, and Indradeep Ghosh. 2014. SymJS: automatic symbolic testing of JavaScript web applications. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering* (Hong Kong, China) (FSE 2014). Association for Computing Machinery, New York, NY, USA, 449–459. <https://doi.org/10.1145/2635868.2635913>
- [9] Zhe Li and Fei Xie. 2023. In-Situ Concolic Testing of JavaScript. In *2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 236–247. <https://doi.org/10.1109/SANER56733.2023.00031>
- [10] Blake Loring, Duncan Mitchell, and Johannes Kinder. 2017. ExpoSE: practical symbolic execution of standalone JavaScript. In *Proceedings of the 24th ACM SIGSOFT International SPIN Symposium on Model Checking of Software* (Santa Barbara, CA, USA) (SPIN 2017). Association for Computing Machinery, New York, NY, USA, 196–199. <https://doi.org/10.1145/3092282.3092295>
- [11] Blake Loring, Duncan Mitchell, and Johannes Kinder. 2019. Sound regular expression semantics for dynamic symbolic execution of JavaScript. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation* (Phoenix, AZ, USA) (PLDI 2019). Association for Computing Machinery, New York, NY, USA, 425–438. <https://doi.org/10.1145/3314221.3314645>
- [12] Aleksei Menshutin. 2025. Path-Minimal Objects in ArkTS Symbolic Execution: Artifact. <https://doi.org/10.5281/zenodo.17659274> Reproducible artifact for the ICCQ 2025 paper “Path-Minimal Objects in ArkTS Symbolic Execution”.
- [13] José Fragoso Santos, Petar Maksimović, Théotime Grohens, Julian Dolby, and Philippa Gardner. 2018. Symbolic Execution for JavaScript. In *Proceedings of the 20th International Symposium on Principles and Practice of Declarative Programming* (Frankfurt am Main, Germany) (PPDP ’18). Association for Computing Machinery, New York, NY, USA, Article 11, 14 pages. <https://doi.org/10.1145/3236950.3236956>
- [14] Prateek Saxena, Devdatta Akhawe, Steve Hanna, Feng Mao, Stephen McCamant, and Dawn Song. 2010. A Symbolic Execution Framework for JavaScript. In *2010 IEEE Symposium on Security and Privacy*. 513–528. <https://doi.org/10.1109/SP.2010.38>
- [15] Koushik Sen, Swaroop Kalasapur, Tasneem Brutch, and Simon Gibbs. 2013. Jalangi: a selective record-replay and dynamic analysis framework for JavaScript. In *Proceedings of*

the 2013 9th Joint Meeting on Foundations of Software Engineering (Saint Petersburg, Russia) (ESEC/FSE 2013). Association for Computing Machinery, New York, NY, USA, 488–498. <https://doi.org/10.1145/2491411.2491447>

- [16] UnitTestBot Team. 2023. USVM: Universal Symbolic Virtual Machine. <https://github.com/UnitTestBot/usvm>. Language-agnostic core for symbolic execution.

A An Example of Generated Tests

For the function `f3b` from Listing 2, our generator produces tests for three terminal paths: (i) an exceptional path when the whole argument is undefined; (ii) the `return 1` path with the path-minimal object `{}`; (iii) the `return 2` path.

The second test uses the path-minimal object `{}` to drive `Number.isNaN(o.x + 1)` to true, while the third illustrates a non-NaN path. The first snippet shows the exceptional path when the function is invoked with undefined.

```
// This helper is used here only to shorten the listing.
// In the actual generator, this boilerplate is inlined per
// test (no shared helper).
const __has = Object.prototype.hasOwnProperty;
const __call = (arg:any) =>
  (typeof Example !== 'undefined'
   && typeof Example.prototype !== 'undefined'
   && __has.call(Example.prototype, 'f3b'))
  ? (new Example()).f3b(arg)
  : Example.f3b(arg);

test('Example.f3b', () => {
  expect(__call(undefined)).toEqual(/* exception */);
});

test('Example.f3b', () => {
  expect(__call({})).toEqual(1.0);
});

test('Example.f3b', () => {
  expect(__call({ x: null })).toEqual(2.0);
});
```

Listing 8. Auto-generated tests for `Example.f3b` (condensed).